

Article

Probing Emergent World Representations in Go Life-and-Death Problems

Zhikai Yang ^{1,2} , Zhigang Meng ^{2,3,*} and Zhiqiang Wen ¹

¹ School of Computer Science and Artificial Intelligence, Hunan University of Technology, Zhuzhou 412007, China; m23077500015@stu.hut.edu.cn (Z.Y.); wenzhiqiang@hut.edu.cn (Z.W.)

² Hunan Provincial Video Cultural and Creative Research Base, Changsha 411233, China

³ School of Computer Science and Engineering, Changsha University, Changsha 410022, China

* Correspondence: z20110956@ccsu.edu.cn

Abstract

Large language models (LLMs) have demonstrated remarkable capabilities in learning complex tasks purely from sequential data. To explore whether such models can internalize strategic world representations, We investigate whether generative transformer models can learn structured world representations from sequential data. Using the domain of Go life-and-death problems as a controlled micro-world, we train a GPT-style generative model to predict moves from serialized board states. Focusing on localized life-and-death (tsumego) scenarios, we train the model to predict valid next moves from serialized board states without providing any explicit Go rules or strategic supervision. Probing the model's internal activations reveals structured representations aligned with liberties, eyes, and tactical group status. To interpret these representations, we introduce the Multi-Aspect World Probe (MAWP), a modular probing framework that disentangles tactical concepts into orthogonal dimensions. We further apply interventional techniques to manipulate internal representations and causally evaluate their impact on model predictions. Our results show that the proposed model achieves 94.7% accuracy in sequence correctness and 92.1% in outcome validity on life-and-death tasks. This work extends interpretability research into spatially structured domains and offers tools for understanding decision-making in sequence models.

Keywords: generative AI; causal intervention; transformer models; emergent representations; large language models (LLMs)

1. Introduction

In the study of Go life-and-death problems, traditional approaches predominantly rely on carefully designed heuristic rules and search strategies to efficiently capture local tactical structures. For instance, Benson's Algorithm [1] determines the unconditional life status of a group by identifying true eye structures, thereby providing a theoretically rigorous criterion for safety. Heuristic-based life-and-death analysis methods, such as those proposed by Kraszek [2], guide the search process by summarizing recurring local patterns, including eye shapes, connectivity, and liberty configurations. Furthermore, Martin Müller [3] and colleagues introduced safe-solving techniques that integrate pruning strategies with localized region analysis, effectively reducing the complexity of the search space. In the context of capture problems, David Chen [4] and others developed heuristic search methods that improve efficiency by prioritizing critical liberties and urgent responses. More advanced search frameworks, such as Lambda Search [5], perform



Academic Editor: Isidoros Perikos

Received: 10 March 2026

Revised: 22 April 2026

Accepted: 7 May 2026

Published: 14 May 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

recursive analysis within localized regions to explore potential life-and-death variations, enabling more refined tactical reasoning. In recent years, Relevance-Zone-Based Search [6] has further improved efficiency by dynamically constraining the search space to regions most relevant to the current tactical situation, thereby focusing computational resources while maintaining high solution accuracy.

Recent advances in generative AI have demonstrated that large-scale transformer models can learn complex structured representations from sequential data. The advent of AlphaGo [7] marked a new era in artificial intelligence, demonstrating that the combination of deep reinforcement learning (RL) and Monte Carlo Tree Search (MCTS) [8] can surpass top human experts in the ancient game of Go. This success was soon generalized and simplified in AlphaZero [9], and subsequent agents such as KataGo [10] and Leela Zero [11] have achieved levels far beyond professional players. Despite their superhuman performance, these agents are optimized purely to win games and are trained in an end-to-end fashion. As a result, they typically lack explicit internal representations of key substructures in the game, such as capturing, ko fights, or life-and-death judgments.

Directly applying LLMs to play full games of Go poses significant challenges [12]. The life-and-death status of certain stones often depends on complex long-range interactions, which are difficult for LLMs to capture reliably due to the entangled nature of local and global dependencies. Moreover, in whole-board prediction tasks, LLM decisions often lack interpretability and fail to express clear subgoals (e.g., “make this group live” or “initiate a ko fight”), making diagnostic analysis difficult. This motivates our focus on building a lightweight model dedicated to solving life-and-death problems. These problems typically involve only a local region of the board, bounded by closed borders, allowing the model to concentrate on precise reasoning rather than global estimation. Due to their small subspace and singular objectives, the model’s behavior is easier to analyze and visualize, enabling the generation of saliency maps, probe plots, and other explanatory structures to investigate latent tactical patterns in Go.

Inspired by this perspective, we propose to re-examine Go modeling from a structured standpoint. Rather than predicting the final outcome of the game, we explore whether a large-scale Transformer model trained purely on sequential data from life-and-death situations can learn to internalize the underlying tactical structures of Go.

LLMs trained on next-token prediction tasks [13] have demonstrated surprising abilities to solve problems far beyond their training data, including logical reasoning, symbolic manipulation, and strategic planning [14]. A growing body of work suggests that such capabilities may stem from emergent internal representations that reflect the latent structure of the tasks they are exposed to [15], even when those tasks involve no explicit symbolic grounding.

In this work, we extend the investigation of emergent world representations to the domain of Go, specifically focusing on life-and-death localized tactical puzzles that determine the survival of stone groups. Inspired by prior work on Othello-GPT [16], which demonstrated the emergence of internal board state representations in a transformer model trained on legal Othello move sequences [17], we ask can similar structures emerge in Go without explicit rule encoding or symbolic supervision? To answer this question, we train a GPT-style model to predict moves in carefully planned life-and-death problems using board positions and state sequences as input.

We employ nonlinear probes to examine whether the model internally encodes group liberties, eyes, and local board status. Existing probing approaches primarily focus on decoding individual features from model representations using linear or nonlinear classifiers. While such methods can demonstrate that certain information is present in hidden states, they suffer from two key limitations. First, they do not capture the compositional

nature of tactical reasoning in Go, where multiple interdependent factors jointly determine outcomes. Second, standard probes treat these features independently, which may overlook structured relationships among them and limit interpretability. To address these limitations, we propose MAWP, which explicitly models multiple tactical features in a unified framework. Instead of treating each feature independently, MAWP decomposes complex decision factors into a set of complementary aspects and learns their joint contributions to the model's predictions. This design better aligns with the structured and compositional nature of life-and-death reasoning, enabling more faithful and interpretable analysis of internal representations. Further, we introduce interventional techniques to manipulate internal activations and measure the causal effect on move predictions. These interventions allow us to not only test the model's reliance on latent game state representations but also generate latent saliency maps that attribute strategic significance to individual stones or regions.

By demonstrating the emergence and causal utility of internal Go representations, our work extends the scope of interpretability research into high-dimensional, strategic, and spatial domains, and provides tools for analyzing decision-making processes in general-purpose sequence models.

The Life-and-Death Problems of Go as a Testbed for Interpretability

Building on the idea of using games as interpretable micro-worlds, we propose Go life-and-death problems as an effective testbed for studying world representations in sequence models. These problems aim to determine whether a local group of stones can survive under perfect play, offering a constrained yet strategically rich environment.

Compared to full-board games, life-and-death problems are more tractable, involving only a small region and limited move space. This allows us to isolate critical local concepts such as liberties, eyes, and vital points—core to Go strategy but difficult to infer from raw sequences. Following the approach of Othello-GPT, we train a transformer model to predict optimal moves without rule supervision, using only sequences of local board states (see Figure 1).

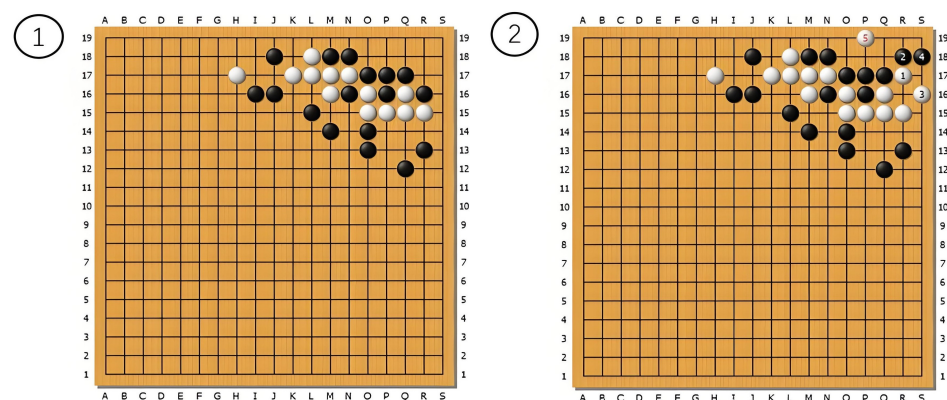


Figure 1. Go life-and-death examples. (1) The original configuration of a crucial Go life-and-death problem. (2) The corresponding solution variation, where the numbered markers indicate the sequential order of key moves used to resolve the problem. In Go, players alternately place black and white stones, typically with Black moving first. A group is considered alive if it can form two eyes; otherwise, it is considered dead.

A key question in our study is whether the model develops internal representations that reflect the current state of a local Go life-and-death configuration. Specifically, we ask whether such a world state representation exists within the model and plays a role in predicting the next move. To investigate this, we employ a probing approach. We train a

set of classifiers—referred to as probes—that take the model’s internal activations as input and attempt to recover the corresponding board state. Each probe is trained to predict, for every position in the local board, whether it is occupied by black or white or is empty. This allows us to assess whether the internal representations encode sufficient information about group structure and liberties. We adapt this framework to the Go domain, where latent spatial representations are essential for understanding life-and-death dynamics.

To sum up, we present three contributions: (1) We propose Go life-and-death problems as a new, interpretable testbed for analyzing internal representations in transformer models. (2) We provide evidence that a GPT-style model trained solely on local board sequences can learn emergent representations aligned with core Go concepts such as liberties and group life status. (3) We propose a probing framework, Multi-Aspect World Probe, to analyze the internal representations of life-and-death, which is better suited to life-and-death and has more advantages over previous probing methods.

2. Language Modeling of Go Life-and-Death Sequences

Inspired by the fact that Transformer models can learn effective chess or Othello moves by simply observing game records [18], we explore whether similar strategies are applicable to localized Go life-and-death problems. We choose the life-and-death scenario instead of the full Go game because it is local and maintains high complexity.

Our goal is to test what LifeGo-GPT can learn about survival and capture by observing the sequence of moves in these local subgames without knowing the rules of Go. The game board is a 19×19 basic Go board. Each game has an initial situation, and most of the time the black player has the first move. Each life-and-death has a unique move sequence. A piece has two or more true eyes (independent empty points that the opponent cannot fill at the same time). Even if it is surrounded, it cannot be captured, and is called a live piece. The following describes the training process of the model.

2.1. Go Datasets

The dataset is collected from multiple publicly available Go resources, including online platforms such as 101 Weiqi, Xingzhen Go Platform, and Jueyi Go Platform, as well as curated problem sets and classical literature, such as Weiqi Life-and-Death Encyclopedia and Fayang Lun. All selected problems correspond to standard life-and-death (tsumego) scenarios and are accompanied by verified solution sequences. The dataset spans a wide range of difficulty levels, from single-move survival problems to complex multi-step tactical sequences. Easy problems typically involve localized corner situations with small groups and limited liberties, where the outcome is determined by straightforward capturing tactics and can be resolved in only a few moves. Medium-difficulty problems introduce capturing races and eye-space competition, where multiple groups interact and offensive and defensive roles may shift dynamically; these problems often involve multiple plausible variations and require strategic choices between forming and destroying eyes. Hard problems, in contrast, feature large-scale entangled group fights with multiple potential life patterns and attack routes. The number of variations grows combinatorially, often requiring long-horizon reasoning and, in some cases, global positional judgment. The approximate distribution of difficulty levels in the dataset is 5:3:2 for easy, medium, and hard problems, respectively. Although some problems admit multiple valid solutions, we adopt the canonical solution sequence provided in the source material as the reference. In outcome-based evaluation, alternative solution sequences that lead to the same final status are also considered correct. The original data is in sgf format, and we process these SGF files into flat sequences suitable for autoregressive modeling shown in Figure 2. Each data instance contains two parts: (1) the initial local board layout centered on the target stone

piece group; (2) the solution sequence. The data is serialized into a token sequence that encodes both the spatial layout and the move history.

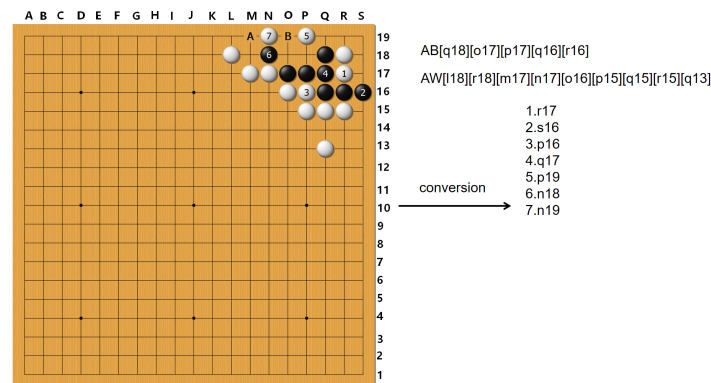


Figure 2. In the figure, we convert the stoneboard representation of life-and-death into a text sequence.

To ensure a fair and reliable evaluation, we performed dataset deduplication at the puzzle level prior to any data augmentation. Exact duplicate samples were removed, and symmetry-equivalent board states under rotations and reflections were canonicalized to identify and eliminate redundant instances. The dataset was then split into training and validation sets at the original puzzle level, ensuring that all augmented variants of a given puzzle remain within the same split. Furthermore, we restricted augmentation operations to transformations that preserve the semantics of life-and-death tasks. In particular, color inversion was applied only when the task definition remained invariant under player color exchange; otherwise, it was excluded. We also conducted an audit of near-duplicate samples to minimize the risk of validation leakage and artificially inflated accuracy. The dataset contains about 10,000 life-and-death scenarios, divided into 8:2 training and validation sets to train our model.

2.2. Task-Specific Autoregressive Modeling

Although our backbone follows a GPT-style decoder-only Transformer [19], the implementation is tailored to the Go life-and-death task [20]. Each tsumego instance is converted from SGF format into a serialized token sequence that contains two components: (i) the local board configuration centered on the target group and (ii) the reference solution sequence. For a standard 19×19 board, each intersection is mapped to a unique coordinate token, yielding a coordinate vocabulary of size 361. Although board coordinates are mapped to integer token IDs (1–361), this indexing does not impose an explicit spatial inductive bias in the model. Specifically, token IDs are treated as categorical indices that are mapped to learned embeddings without any ordinal or geometric constraints. We note that while the numerical ordering of token IDs could, in principle, introduce a weak implicit structure, the embedding layer is randomly initialized and learned independently, which prevents the model from directly exploiting ordinal relationships between token indices. We additionally introduce a small set of special tokens, such as <BOS>, <SEP>, and <EOS>, to separate the board state from the move sequence and to mark sequence boundaries.

Let $\mathbf{x} = (x_1, x_2, \dots, x_n)$ denote the serialized input tokens of a problem, where the prefix encodes the current board state and the suffix encodes the canonical solution moves. Given the token sequence, the model is trained in an autoregressive manner to predict the next token at each time step:

$$p_{\theta}(x_t \mid x_{<t}) = \text{softmax}(W_{\theta}h_t), \quad (1)$$

where h_t is the hidden representation produced by the final Transformer layer at position t , and W_o is the output projection matrix.

2.2.1. Input Representation

To preserve the spatial structure of the Go board while remaining compatible with sequence modeling, we flatten the local board region into a token stream. Each intersection is encoded according to its occupancy state, namely black, white, or empty, together with its coordinate identity. The serialized input therefore retains both the geometric layout and the tactical context of the local position. This design allows the model to learn spatial regularities from data alone, without any explicit rule-based supervision.

2.2.2. Output Target and Training Objective

The training target is the canonical move sequence provided by the source materials. Although some tsumego instances may admit multiple valid solution paths, we use the reference solution as the supervised target during training. During evaluation, alternative move sequences that lead to the same life-and-death outcome are also considered correct.

The model is optimized with the standard next-token prediction objective under teacher forcing:

$$\mathcal{L}_{\text{LM}} = - \sum_{t=1}^n \log p_{\theta}(x_t \mid x_{<t}). \quad (2)$$

This objective encourages the model to learn the sequential regularities of life-and-death play, including vital-point selection, eye formation, capturing races, and local tactical forcing moves.

2.2.3. Architecture and Optimization

Table 1 shows that we instantiate LifeGo-GPT as an 8-layer causal Transformer with 8 attention heads and a hidden size of 512. All parameters, including token embeddings and positional embeddings, are randomly initialized. Training is performed with the Adam optimizer using a learning rate of 10^{-4} and a batch size of 64 for 100 epochs. We apply gradient clipping with a maximum norm of 1.0 to stabilize optimization. Early stopping is selected according to validation performance. To evaluate the stability of the training process, we plot the training and validation loss curves, as shown in Figure 3. The model exhibits a smooth and consistent decrease in loss during training, indicating effective optimization under the chosen hyperparameter settings.

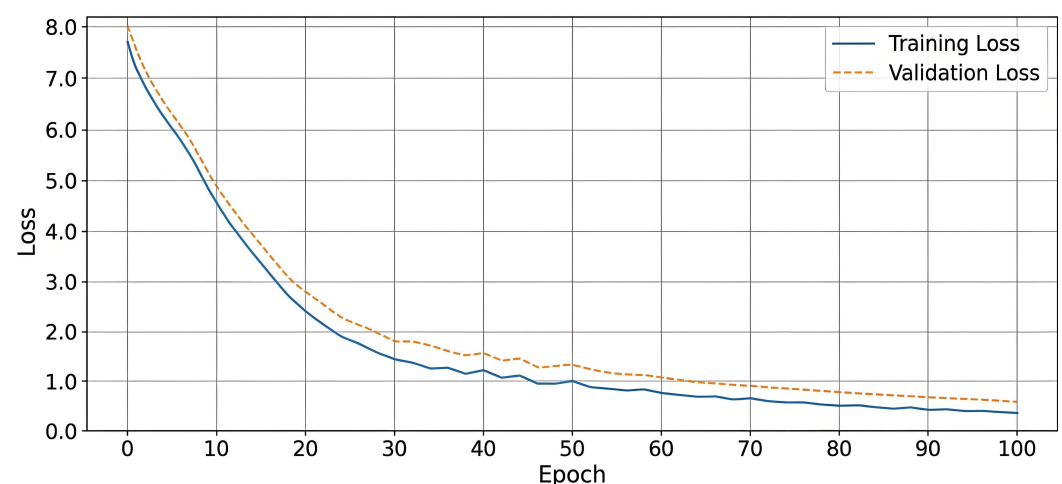


Figure 3. Training loss curve of LifeGo-GPT on the training set.

Table 1. Training hyperparameters of the proposed LifeGo-GPT model.

Hyperparameter	Value	Description
Number of layers (L)	8	Depth of the Transformer encoder
Attention heads	8	Number of heads in multi-head attention
Hidden dimension (d)	512	Dimensionality of hidden representations
Optimizer	Adam	Adaptive gradient-based optimization method
Learning rate (η)	1×10^{-4}	Step size for parameter updates
Batch size	64	Number of samples per training batch
Training epochs	100	Total number of training iterations over dataset
Gradient clipping	Max norm = 1.0	Stabilizes training by limiting gradient magnitude

2.2.4. Decoding and Legality Handling

At inference time, the model generates moves autoregressively from the serialized board state. The predicted coordinate token is mapped back to the corresponding board intersection. For legality-based evaluation, a prediction is counted as legal only if the selected move is valid under the current board state. For sequence-level evaluation, we compare the generated solution with the canonical reference sequence, while also allowing alternative sequences that preserve the same final life-and-death outcome. Algorithm 1 summarizes the task-specific training pipeline.

Algorithm 1 Training LifeGo-GPT on Go Life-and-Death Sequences

Require: Dataset of tsumego problems $\mathcal{D} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^N$, where $\mathbf{x}^{(i)}$ denotes the serialized board state and $\mathbf{y}^{(i)}$ denotes the reference solution sequence

Require: Transformer model f_θ , learning rate η , number of epochs E

```

1: for  $e = 1$  to  $E$  do
2:   for each problem  $(\mathbf{x}, \mathbf{y}) \in \mathcal{D}$  do
3:     Construct the full input sequence by concatenating the board-state tokens and
       the solution tokens
4:     Compute the hidden states by forwarding the sequence through  $f_\theta$ 
5:     Compute the next-token distribution at each position
6:     Evaluate the autoregressive cross-entropy loss
7:     Update parameters using Adam:
8:        $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\text{LM}}$ 
9:   end for
10:  Evaluate the model on the validation set
11:  Apply early stopping if validation performance does not improve
12: end for
13: return  $\theta$ 

```

2.3. Legal Move Prediction with LifeGo-GPT

To assess the model's ability to internalize life-and-death concepts, we define three evaluation metrics: Move Legality: whether the predicted move corresponds to a valid legal position on the board; Sequence Correctness: whether the model's predicted sequence matches the canonical solution from exercise collections; Outcome Validity: whether the predicted sequence leads to the correct life-and-death outcome, regardless of whether it matches the canonical solution sequence.

Although a canonical solution sequence is provided in the dataset, it is not the only valid solution. In many life-and-death problems, multiple distinct move sequences may lead to the same final outcome. Therefore, we distinguish between sequence-level correctness and outcome-level validity in our evaluation. They are counted as incorrect under sequence correctness, which requires an exact match to the canonical solution. To compute outcome validity, we simulate the final board state obtained after executing the predicted move sequence. The life-and-death status of the target group is then determined using an

external Go engine KataGo as a verifier. The predicted sequence is considered valid if the resulting status matches the ground-truth outcome label.

We compare LifeGo-GPT's performance with several established baselines and algorithms for solving life-and-death problems (see Table 2): an untrained transformer (random initialization), and existing Go engines such as KataGo and Leela Zero. Since these engines are optimized for full-board competitive play, comparisons are restricted to localized tsumego scenarios to ensure fairness.

Table 2. Comparison of LifeGo-GPT with baselines and Go engines on localized life-and-death problems.

Method	Move Legality (%)	Sequence Correctness (%)	Outcome Validity (%)
Untrained Transformer	15.8	12.3 ± 2.1	10.5
LifeGo-GPT (ours)	98.6	94.7 ± 0.6	92.1
KataGo (local only)	99.2	96.5 ± 0.4	97.8
Leela Zero (local only)	98.9	92.4 ± 0.7	90.7
RZS	88.6	78.3 ± 1.5	72.4
ANN	96.3	94.5 ± 0.9	89.4

After training, LifeGo-GPT achieves strong performance on localized life-and-death problems. On the validation set, the model reaches 94.7% correctness in Sequence Correctness and 98.6% accuracy in Move Legality, substantially higher than the untrained baseline (15.8%). Interestingly, the model learns to respect basic tactical rules, such as avoiding self-atari and seeking vital points, despite never being explicitly taught these rules. Compared with engines like KataGo, LifeGo-GPT performs competitively on localized tsumego tasks, though professional engines remain stronger in more complex, full-board scenarios.

We further conduct a failure case analysis to better understand the limitations of the proposed model. We observe that errors mainly occur in three scenarios: (1) complex life-and-death problems requiring multi-step reasoning, (2) long solution sequences with deeper tactical dependencies, (3) ambiguous positions where multiple moves yield similar outcomes.

These findings suggest that while the model captures local tactical patterns effectively, it still faces challenges in handling long-range dependencies and highly uncertain situations.

3. Multi-Aspect World Probe: Capturing Multi-Faceted Life-and-Death Semantics

Life-and-death reasoning in Go is inherently multifaceted. Determining whether a group of stones is alive, dead, or unsettled requires understanding a variety of interdependent tactical concepts—such as liberties, eyes, group connectivity, and ko dynamics. Single-label probing methods obscure which tactical features are internally represented, and how they contribute to predictions.

To address this, we propose the MAWP: a probing framework that disentangles tactical semantics into orthogonal aspects, each corresponding to a specific concept relevant to life-and-death analysis. Instead of treating survival status as a monolithic label, MAWP decomposes it into interpretable dimensions—such as liberty count, eye structure, and ko involvement—that can be probed, visualized, and intervened upon independently.

3.1. Probing Architecture

The MAWP is a modular probing system designed to extract interpretable tactical concepts from the hidden representations of LifeGo-GPT. As illustrated in Figure 4, MAWP consists of a shared feature extractor followed by five parallel classification heads, each corresponding to a distinct tactical aspect relevant to life-and-death reasoning.

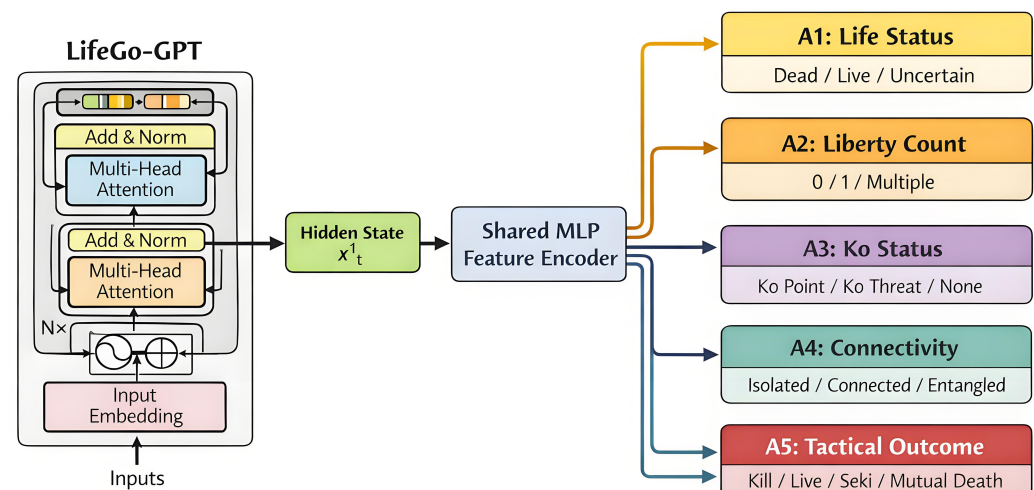


Figure 4. This figure shows the structure of MAWP.

3.1.1. Shared Feature Encoder

At each time step t , the transformer's hidden state x_t^l from a selected layer l is passed through a shared multi-layer perceptron (MLP). This shared encoder learns a compact latent representation that captures task-relevant semantic structure while preserving a common embedding space across all probing tasks.

3.1.2. Aspect-Specific Heads

The output of the shared MLP is then routed to five classification heads, each responsible for predicting one aspect of the tactical world state:

- **A1: Life Status**—Classifies each tile as belonging to a dead group, a live group, or one whose status is undetermined.
- **A2: Liberty Count**—Estimates the number of liberties for a stone or group: zero, one, or multiple.
- **A3: Ko status**—Labels tiles based on their involvement in ko situations: ko point, ko threat, or irrelevant.
- **A4: Connectivity**—Determines whether a stone is isolated, part of a stable connection, or entangled in a potentially dead group.
- **A5: Tactical Outcome**—Categorizes the local structure as resulting in kill, live, seki, or mutual death.

Each of these heads receives input from a shared MLP-based feature extractor and outputs predictions independently. The resulting aspect-wise predictions can be visualized and interpreted individually or jointly to offer insight into the model's internal understanding.

3.2. Training Procedure

Given hidden representations x_t^l from the l -th layer of the LifeGo-GPT:

$$x_t^{(l)} \in \mathbb{R}^{N \times d} \quad (3)$$

where N is the number of local board grid points (e.g., 19×19 or sub-region) and d is the hidden dimension. MAWP applies a shared feature extractor $\phi(x)$ followed by per-aspect classification heads:

$$h_i(\phi(x)) = \text{softmax}(W_i \phi(x)) \quad \text{for } i = 1, \dots, 5 \quad (4)$$

And MAWP applies to each tile:

$$z_{t,p} = \phi(x_{t,p}^{(l)}), \quad p \in \{1, \dots, N\} \quad (5)$$

$$\hat{y}_{t,p}^{(i)} = h_i(z_{t,p}) \quad (6)$$

The overall probing loss is defined as a weighted sum of the individual cross-entropy losses:

$$\mathcal{L}_{\text{probe}} = \sum_{i=1}^5 \lambda_i \cdot \mathcal{L}_{\text{CE}}(h_i(\phi(x)), y^{(i)}) \quad (7)$$

where $y^{(i)}$ denotes the ground-truth labels for aspect i , and λ_i represents weighting coefficients.

We adopt the uniform weighting scheme by default, setting $\lambda_i = 1/5$ for all five aspects. This simple but effective approach assumes that each semantic probe contributes equally to the model's understanding of the board state. While more sophisticated weighting strategies (e.g., based on task difficulty or label imbalance) are possible, we find that uniform weighting yields stable and interpretable results across a wide range of life-and-death problems.

Ground-truth labels for each aspect are generated automatically using KataGo-based analysis and explicit post-processing rules defined for each tactical concept. MAWP offers several advantages: multi-aspect interpretability, improved supervision and better generalization.

Automatic Label Construction

To make the probing tasks reproducible, we explicitly define how each semantic label is derived from the board state and engine verification results. For each local tsumego position, we first identify the target stone group and compute its connected component on the board. The liberty count is obtained by enumerating the distinct empty intersections adjacent to that component. The life-status label is then assigned by verifying the position with a strong Go engine under a fixed search budget. We map the engine verdict to a discrete set of labels, including alive, dead, seki, and unresolved. Positions with inconsistent or ambiguous engine outputs are excluded from the probing dataset.

For the remaining aspects, we use deterministic local rules whenever possible. Connectivity is defined over same-color stones under 4-neighborhood adjacency within the local board region. Ko-related labels are assigned only when the position contains an explicit ko pattern identified by the engine's principal variation and local rule-based verification. The tactical outcome label is determined from the final life-and-death status of the target group after executing the reference solution and re-evaluating the terminal board state.

4. Validating Probes with Interventional Experiments

While probing methods reveal that tactical information such as liberties, life status and connectivity can be decoded from internal representations, this alone does not imply that the model explicitly relies on these features as decision variables. In particular, probes with sufficient capacity may recover information that is only correlationally present in hidden states, rather than being causally involved in the model's prediction process. To move beyond purely observational analysis, we introduce interventional experiments [21] aimed at assessing the causal relevance of these representations. Rather than claiming definitive causal identification, our goal is to examine whether targeted modifications to semantically meaningful internal features can systematically influence model behavior [22].

Intervention Method

We first created an evaluation benchmark set, which consists of the initial positions of black and white stones, the target positions of the intervention, and the correct move sequences. The benchmark set contains 1000 life-and-death cases, making subsequent interventions more applicable to life-and-death. We intervened in the hidden state of LifeGo-GPT by selecting a target position x_t^l (usually the center of the critical group) and apply a transformation to the hidden representation that corresponds to a change in the semantic attribute of the target position, such as simulating a color flip. In this work, all interventions are performed in the representation space. The description of flipping a stone refers to a semantic interpretation of the transformation applied to the hidden state, rather than a direct modification of the input.

This operation was performed at layer $l = 6$. In the lower layers, the model's activations are dominated by local pattern extraction and surface-level token interactions, so interventions there tend to perturb generic structural features rather than the higher-order game-state representations of interest. In the upper layers, by contrast, representations become increasingly specialized for the final prediction target, which makes causal effects harder to interpret because they are already tightly coupled to the output decision. Layer 6 lies near the middle of the network, where intermediate representations are typically most suitable for capturing compositional information such as board state, tactical context, and emerging semantic structure while still retaining enough flexibility to reflect causal changes. For this reason, intervening at Layer 6 is more likely to reveal meaningful propagation of internal state changes, whereas intervening at neighboring layers may either be too early to influence the relevant abstractions or too late to separate representation from decision-making.

After intervention, we fed the modified board state into the remaining Transformer layers and recorded the new move logit value:

$$\tilde{x}_t^{(l)} = \text{do}(x_{t,g}^{(l)} \leftarrow \psi(x_{t,g}^{(l)}, c)) \quad (8)$$

where g represents the position of the stoneboard being interfered with and ψ represents a transformation in representation space that simulates a change in the semantic attribute. The detailed steps are presented in Algorithm 2. We compared this result with the logit value pre-intervention to assess how much the model's predictions changed due to the injected internal features:

$$\Delta a = f_\theta(\tilde{x}_t^{(l)}) - f_\theta(x_t^{(l)}) \quad (9)$$

Importantly, this intervention is targeted rather than random: it focuses on positions that are semantically meaningful in the life-and-death context. This design aims to selectively perturb latent features associated with tactical concepts, rather than introducing arbitrary noise.

Algorithm 2 Intervention on Go Board for Life-and-Death

Inputs: Initial board state B (positions of black and white stones); legal next moves a ; target position for intervention g ; post-intervention color c .

Output: $\{L_l\}_{s \in B}$: the logits difference values for each board state.

```

1: Initialize the Go board state  $B$  with current black and white stones positions;
2: Determine the set of all legal next moves  $a$  under the current state  $B$ ;
3: Specify the target position  $g$  for intervention;
4: Specify the color  $c$  after intervention at position  $g$ ;
5: Compute the original next-step logits without intervention:
6:    $a_0 \leftarrow f_a(x_{t-1})$ ;
7: Initialize an empty set for storing logits differences  $\{L_l\}$ ;
8: for each board state  $s \in B$  do
9:   Apply intervention at the target position:
10:   $\tilde{x}_{t-1} \leftarrow \text{Intervene}(x_{t-1}, g, c, s)$ ;
11:  Perform forward inference on the intervened state:
12:   $a_s \leftarrow f_a(\tilde{x}_{t-1})$ ;
13:  Compute the logits difference between original and intervened states:
14:   $L_l \leftarrow a_0 - a_s$ ;
15:  Store  $L_l$  into the result set;
16: end for
17: return  $\{L_l\}_{s \in B}$ ;

```

5. Ablation Study

5.1. Results and Analysis

Figures 5 and 6 shows the pre-intervention and post-intervention logits for a representative example. The left panels illustrate the board state, and the right panels show predicted move logits. In this example, we injected a representation of a live group into a position previously associated with low survival likelihood.

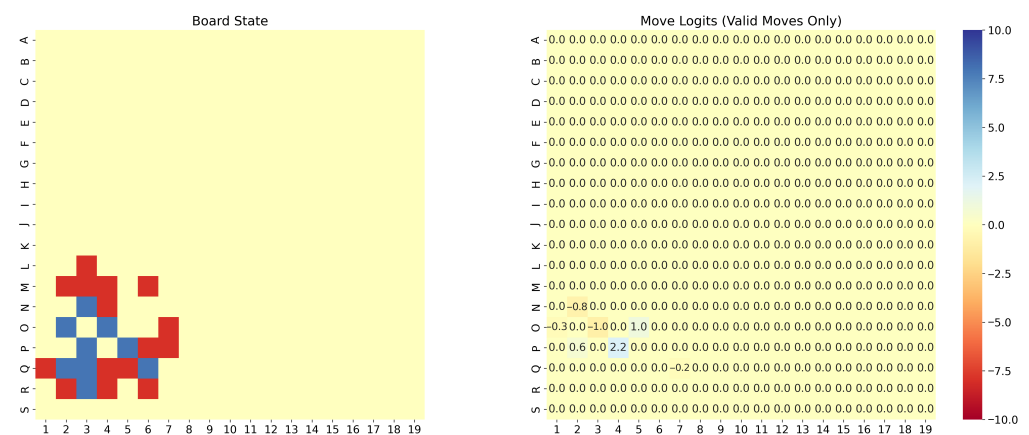


Figure 5. The game state and model prediction before intervention. The **left** panel shows the local board configuration of the life-and-death problem. The **right** panel visualizes the predicted move logits over the board. Warmer colors correspond to lower predicted likelihoods, while cooler colors indicate higher scores. High-response regions highlight the model's preferred candidate moves prior to any intervention.

Post-intervention, the model's predictions exhibit a noticeable shift. While the logits associated with key points do not decrease significantly, the high-scoring regions (blue) move toward more stable outer positions. This pattern suggests a systematic change in the model's tactical preferences, consistent with the influence of the intervened representations. While the intervention causes changes in the predicted moves, we acknowledge that not all changes necessarily correspond to optimal or globally consistent strategies. However, we

observe that the majority of post-intervention predictions remain legal moves, suggesting that the model does not degenerate into random behavior. Instead, the intervention primarily alters the relative preference among plausible moves.

Quantitatively, across 1000 evaluated cases:

- The top-1 predicted move changes in 95.4% of cases after intervention.
- The mean absolute change in logits over the top-5 positions is 3.21.
- Random noise injections only change top-1 prediction in 5.6% of cases.
- Magnitude-matched random direction perturbations change the top-1 prediction in 18.7% of cases.

Empirically, we find that the top-1 predicted move changes in 95.4% of cases following intervention, with a mean absolute logit shift of 3.21 among the top-5 candidates. In contrast, random perturbations applied to hidden states lead to prediction changes in only 5.6% of cases, perturbations with the same magnitude but random directions induce substantially smaller changes than semantically guided interventions. This pronounced disparity suggests that the observed behavioral shifts are unlikely to arise from arbitrary instability. Instead, they are more plausibly associated with structured internal features. In particular, interventions at semantically meaningful locations induce consistent changes in predicted moves, indicating that these features are functionally connected to the model's decision process.

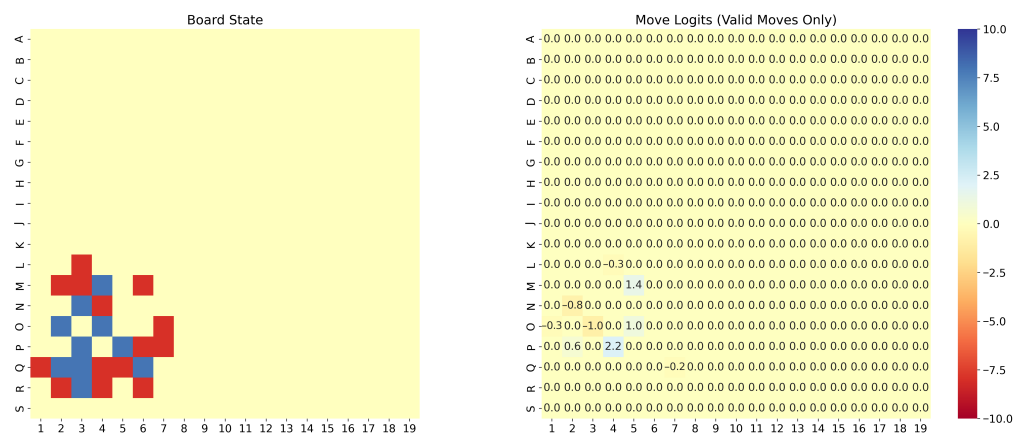


Figure 6. The game state and model prediction after intervention. Compared to the pre-intervention case, the distribution of high-response regions shifts toward more stable outer positions, reflecting a systematic change in the model's tactical preference induced by the intervention.

At the same time, we note that a large prediction shift alone does not establish that a specific semantic variable has been precisely controlled. In principle, a strong sequence model may exhibit high sensitivity to structured perturbations at critical locations, regardless of whether such perturbations correspond to well-defined semantic concepts. While our use of targeted interventions and random perturbation baselines partially mitigates this concern, it does not fully disentangle feature-specific effects from general sensitivity. Consequently, the evidence presented here should be interpreted as indicative of causal relevance, rather than definitive proof that the model explicitly relies on these concepts as decision variables.

Taken together, these results highlight the complementary roles of probing and intervention: probing characterizes what information is encoded in representations, while intervention provides evidence regarding whether such information is functionally utilized during prediction.

To disentangle the effects of architectural design and supervision strategy on model interpretability and probing performance, we conducted a series of ablation studies targeting two key factors: model depth and data augmentation.

5.2. The Impact of Different Probes on Detection Accuracy at Different Model Depths

We begin by examining how the number of transformer layers influences the quality of internal representations, as measured by probe accuracy. To this end, we train several variants of LifeGo-GPT with 2, 4, 6, and 8 layers, and evaluate the performance of linear, nonlinear, and MAWP at each layer.

As shown in Figure 7, probe accuracy consistently improves with model depth across all methods. In particular, MAWP achieves substantially higher accuracy than both linear and standard nonlinear probes, especially in the mid-to-deep layers, where the performance gap becomes most pronounced. This trend suggests that deeper transformer layers progressively encode more structured and semantically meaningful representations. While shallow layers capture low-level token correlations, deeper layers appear to aggregate spatial and tactical information, enabling more accurate decoding of concepts such as liberties and group connectivity.

Moreover, the stronger performance of MAWP compared to single-task probes indicates that disentangling tactical concepts into multiple aspects provides a more effective supervisory signal. By jointly modeling complementary features, MAWP better aligns with the compositional structure of life-and-death reasoning, leading to improved interpretability and predictive accuracy.

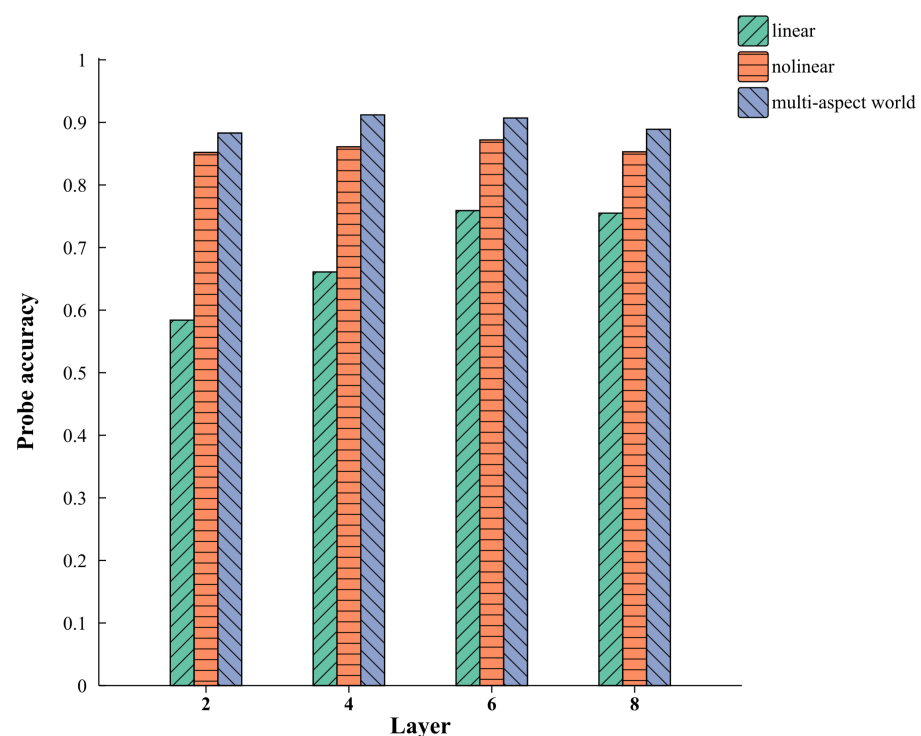


Figure 7. Probe accuracy of MAWP, linear and nonlinear probes under different depth models.

5.3. Effect of Data Augmentation

Figure 8 illustrates the effect of board-level data augmentation on probe accuracy. Removing augmentation consistently degrades performance across all layers and probe types, with the most noticeable drop observed in deeper layers. This result indicates that symmetry-aware augmentation plays a crucial role in encouraging the model to learn invariant representations of Go patterns. Without such augmentation, the model tends

to overfit to specific spatial configurations, resulting in less stable and less generalizable internal representations.

In contrast, augmentation promotes the learning of structure-preserving features that are robust to transformations, which is particularly important in life-and-death problems where equivalent patterns may appear in different orientations. These findings support the conclusion that data augmentation not only improves predictive performance but also enhances the consistency of internal representations.

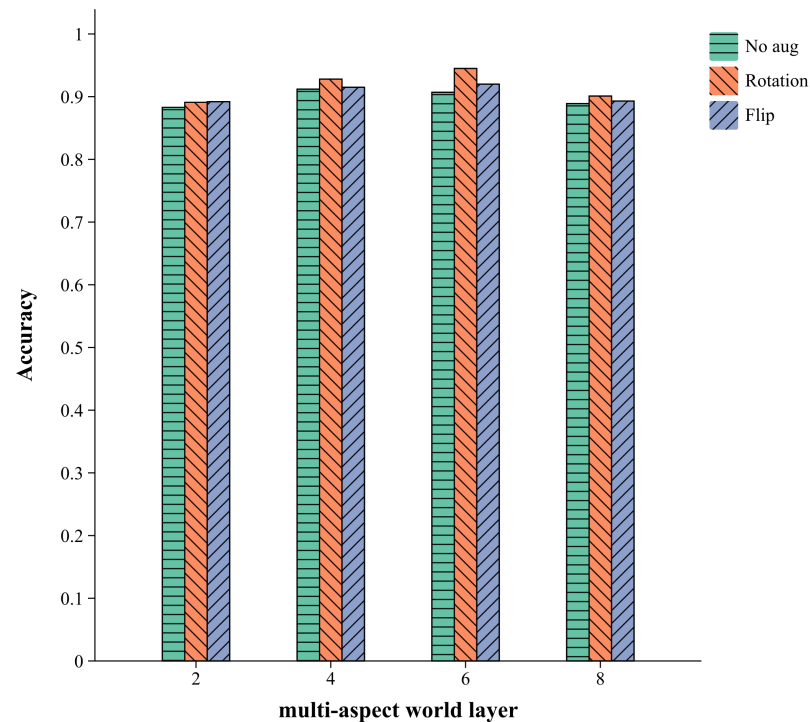


Figure 8. This figure compares the accuracy of models trained with and without stoneboard-level data augmentation at different layers of different classifiers.

6. Conclusions

We propose a new probe framework, which extends the previous work to Go life-and-death problems. Our method and intervention experiments [23] show that GPT-like models can build spatially structured and causally operational representations of Go core concepts [24], which can encode survival-related concepts such as Qi, eyes, and other tactical results, which are not only related to tactical results, but interventions on internal activations can reliably change the model's move decisions. Probing experiments reveal that structured tactical information can be reliably decoded from internal representations. These findings suggest that structured features in representation space play an important role in shaping model behavior. While these results provide evidence for the functional relevance of internal representations, we emphasize that they do not constitute definitive proof of precise semantic variable control. Future work may further explore more controlled causal interventions to isolate feature-specific effects.

While this work focuses on localized life-and-death problems, our findings raise a natural question: to what extent can these emergent representations scale to full-board Go games? In a full-fledged chessboard game, the localized world model ceases to operate as an autonomous decision-making entity; instead, it serves as a tactical-level submodule seamlessly integrated within the global reasoning framework. Initially, the model constructs structured representations of pivotal concepts, including liberties, eye formations, connectivity, and life-and-death statuses, within localized regions. It then decouples and

delineates these tactical elements via a mechanism akin to MAWP. Following this, these localized representations are amalgamated into the global chessboard state encoding, collaborating in the decision-making process alongside global information encompassing situation appraisal, territory estimation, and attack–defense prioritization. Within this framework, the localized world model primarily contributes “state authenticity”, denoting the objective life-and-death attributes of local board configurations. Conversely, the global module undertakes “strategy weighting”, evaluating the strategic significance and optimal timing for addressing this local structure within the broader context of the entire game. Consequently, identical local configurations may elicit disparate decisions under varying global contexts, underscoring that while localized representations exert a causal influence, their ramifications necessitate modulation through the global context. In essence, this hierarchical mechanism of “local tactical modeling + global strategy integration” offers an interpretable avenue for comprehending the multi-scale reasoning prowess of large models in intricate spatial games.

Another promising direction is integrating pretrained multilingual transformers with symbolic game domains such as Go. Recent studies suggest that language models encode latent structural priors, which could potentially enhance decision-making in Go by fusing natural language commentary, game notation, or textual explanations into the input. This opens the door to hybrid models that learn both from board sequences and linguistic supervision, enabling better explainability and generalization, as well as combining board sequences with human commentary or tactical instructions, providing a path to multimodal training. This can enhance model interpretability and performance in ambiguous situations.

Finally, the intervention techniques used here could serve as a blueprint for controllable gameplay agents—allowing human users to guide, debug, or steer neural Go models through interactive feature editing in latent space [25]. We also look forward to someone extending our research to more interesting scenarios.

Author Contributions: Conceptualization, Z.Y. and Z.M.; software, Z.Y.; methodology, Z.W. and Z.Y.; validation, Z.Y. and Z.M.; investigation, Z.Y. and Z.M.; resources, Z.Y. and Z.M.; writing—original draft preparation, Z.Y. and Z.M.; writing—review and editing, Z.M. and Z.W. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Natural Science Foundation of Hunan Province, China (Grant No. 2024JJ7149), Hunan Provincial Video Cultural and Creative Research Base and Scientific Research Fund of Hunan Provincial Education Department in china under (Grant No. 22A0414).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data presented in this study are available in the article. Further information can be obtained from the corresponding author upon request.

Acknowledgments: The authors would like to express their sincere gratitude to Hunan University of Technology, Hunan Provincial Video Cultural and Creative Research Base, and Changsha University for supporting this research.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Benson, D.B. Life in the Game of Go. *Inf. Sci.* **1976**, *10*, 17–29. [[CrossRef](#)]
2. Kraszek, T. Heuristics for the Game of Go. In *Computers and Games: Second International Conference, CG 2000, Hamamatsu, Japan, 26–28 October 2000*; Marsland, T.A., Frank, I., Eds.; Springer: Berlin/Heidelberg, Germany, **2001**; pp. 124–135.
3. Müller, M. Safety of Territory in Games with Applications to Go. *Theor. Comput. Sci.* **1997**, *252*, 147–182.

4. Chen, D.; Müller, M. Automated Go Capture Problems Solving Using Heuristic Search. In *Proceedings of the Computer-Games Workshop (CGW 2002)*; University of Alberta: Edmonton, AB, Canada, 2002; pp. 45–56.
5. Thomsen, T. Lambda-Search in Game Trees—With an Application to Go. *ICGA J.* **2000**, *23*, 203–217. [[CrossRef](#)]
6. Yoshizoe, K.; Kishimoto, A.; Müller, M. A General Product of Successor Selection and Search Area Limitation. In *Computers and Games: 5th International Conference, CG 2006*; van den Herik, H.J., Ciancarini, P., Donkers, H.J.H., Eds.; Springer: Berlin/Heidelberg, Germany, 2007; pp. 168–179.
7. Silver, D.; Huang, A.; Maddison, C.J.; Guez, A.; Sifre, L.; van den Driessche, G.; Schrittwieser, J.; Antonoglou, I.; Panneershelvam, V.; Lanctot, M.; et al. Mastering the game of Go with deep neural networks and tree search. *Nature* **2016**, *529*, 484–489. [[CrossRef](#)] [[PubMed](#)]
8. Liu, P.; Zhou, J.; Lv, J. Exploring the first-move balance point of Go-Moku based on reinforcement learning and Monte Carlo tree search. *Knowl.-Based Syst.* **2023**, *261*, 110207. [[CrossRef](#)]
9. Silver, D.; Hubert, T.; Schrittwieser, J.; Antonoglou, I.; Lai, M.; Guez, A.; Lanctot, M.; Sifre, L.; Kumaran, D.; Graepel, T.; et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science* **2018**, *362*, 1140–1144. [[CrossRef](#)] [[PubMed](#)]
10. Wu, D.J. Accelerating self-play learning in Go. *arXiv* **2019**, arXiv:1902.10565.
11. Leela Zero Team. Leela Zero: A Go Engine with No Human-Provided Knowledge, Modeled after the AlphaGo Zero Paper. Available online: <https://github.com/leela-zero/leela-zero> (accessed on 10 March 2026).
12. Rigaux, T.; Kashima, H. Enhancing Chess Reinforcement Learning with Graph Representation. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*; Neural Information Processing Systems Foundation, Inc.: Vancouver, BC, Canada, 2024; pp. 197–218.
13. Merrill, W.; Goldberg, Y.; Schwartz, R.; Smith, N.A. Provable limitations of acquiring meaning from ungrounded form: What will future language models understand? *Trans. Assoc. Comput. Linguist.* **2021**, *9*, 1047–1060. [[CrossRef](#)]
14. Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. Language models are few-shot learners. *arXiv* **2020**, arXiv:2005.14165. [[CrossRef](#)]
15. Li, B.Z.; Nye, M.; Andreas, J. Implicit representations of meaning in neural language models. *arXiv* **2021**, arXiv:2106.00737. [[CrossRef](#)]
16. Li, K.; Hopkins, A.K.; Bau, D.; Viégas, F.; Pfister, H.; Wattenberg, M. Emergent world representations: Exploring a sequence model trained on a synthetic task. *arXiv* **2022**, arXiv:2210.13382.
17. Choi, S.; Kang, H.; Kim, N.; Kim, J. How does artificial intelligence improve human decision-making? Evidence from the AI-powered Go program. *Strateg. Manag. J.* **2025**, *46*, 1523–1554. [[CrossRef](#)]
18. Shin, M.; Kim, J.; Van Opheusden, B.; Griffiths, T.L. Superhuman artificial intelligence can improve human decision-making by increasing novelty. *Proc. Natl. Acad. Sci. USA* **2023**, *120*, e2214840120. [[CrossRef](#)] [[PubMed](#)]
19. Radford, A.; Narasimhan, K.; Salimans, T.; Sutskever, I. Improving Language Understanding by Generative Pre-Training. *OpenAI Technical Report*. 2018. Available online: <https://openai.com/blog/language-unsupervised/> (accessed on 1 May 2026).
20. Radford, A.; Wu, J.; Child, R.; Luan, D.; Amodei, D.; Sutskever, I. Language Models are Unsupervised Multitask Learners. *OpenAI Technical Report*. 2019. Available online: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf (accessed on 1 May 2026).
21. Qin, L.; Shwartz, V.; West, P.; Bhagavatula, C.; Hwang, J.; Le Bras, R.; Bosselut, A.; Choi, Y. Back to the future: Unsupervised backprop-based decoding for counterfactual and abductive commonsense reasoning. *arXiv* **2020**, arXiv:2010.05906.
22. Krause, B.; Gotmare, A.D.; McCann, B.; Keskar, N.S.; Joty, S.; Socher, R.; Rajani, N.F. GeDi: Generative discriminator guided sequence generation. *arXiv* **2020**, arXiv:2009.06367. [[CrossRef](#)]
23. Dathathri, S.; Madotto, A.; Lan, J.; Hung, J.; Frank, E.; Molino, P.; Yosinski, J.; Liu, R. Plug and play language models: A simple approach to controlled text generation. *arXiv* **2019**, arXiv:1912.02164.
24. Wang, T.T.; Gleave, A.; Tseng, T.; Pelrine, K.; Belrose, N.; Miller, J.; Dennis, M.D.; Duan, Y.; Pogrebniaik, V.; Levine, S.; et al. Adversarial Policies Beat Superhuman Go AIs. In *Proceedings of the 40th International Conference on Machine Learning*; PMLR: Honolulu, HI, USA, 2023; Volume 202, pp. 35655–35739.
25. Schut, L.; Tomašev, N.; McGrath, T.; Hassabis, D.; Paquet, U.; Kim, B. Bridging the human–AI knowledge gap through concept discovery and transfer in AlphaZero. *Proc. Natl. Acad. Sci. USA* **2025**, *122*, e2406675122. [[CrossRef](#)] [[PubMed](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.